Diakrisis — Solution Design Document v4

Project: Real-time risk decision engine for customer banking actions — graduated, explainable friction **Event:** iFX Hack 2026 · 24h · Powered by AWS · Track: Keep Money Safe **Author:** Achilleas Eftychiou **Status:** v4 — final build document. Supersedes v1/v2/v3. Adds the canonical decision-ordering (§8.0) and the deterministic divergence-escalation policy (§8.3). **Build model:** Local-first, built live. AWS documented (§16). Real public datasets only (§5).

---

0. Delta log

1. **Mass payments** (§4A): new MASS_PAYMENT event type with two-level decisions — batch verdict + per-line quarantine. "Quarantine the line, not the payroll."
2. **Stack moved to Java / Spring Boot** (§14): the author's deepest stack and the stack banks actually run. ML via Smile (pure JVM) — no Python anywhere; shared `Features.java` makes train/serve skew impossible by compiler. Java 26 if it runs clean locally tonight, else 25 LTS — zero design impact either way.
3. **Approval workflow** (§8.2): fifth outcome `REQUIRE_APPROVAL` — four-eyes dual control. A coerced victim can self-confirm but cannot self-approve; dual control structurally breaks single-victim coercion.
4. **Hybrid storage with a vector DB** (§9.2, §14): SQLite keeps the exact-key stores; **Qdrant** holds embedded fraud exemplars for k-NN similarity — new signal **M2** + case-based explanations ("resembles 14 confirmed fraud cases"). In-JVM KDTree fallback pre-decided.
5. **ML subsystem upgraded to full treatment** (§9): tuned + calibrated GBT, baseline comparison, full eval suite, feature importances on the ops panel, model card. Resilience principle retained (engine survives ML at weight 0) — that is safety engineering, not simplicity.
6. **Cross-account counterparty reputation** (§4B): a shared, engine-owned store where a HELD/BLOCKED counterparty becomes radioactive for *every other* account in real time — signal **X1** warns the next victim before they finish typing. The one feature that adds a network moat (every Diakrisis bank makes every other safer). Cut-ordered.
7. **ISO 20022 reason codes** (§8.4): each decision emits a standardized machine-readable return/fraud reason code beside the prose — speaks the bank-integration language fluently.
8. **Feedback-loop visibility** (§9.5): cancel = confirmed save, release-after-hold = false-positive; both logged and surfaced as ops counters + an architecture-slide arrow, reframing Diakrisis as a per-bank self-improving system (the answer to "your weights are hand-set").
9. **Vulnerability-aware friction** — documented as a roadmap/pitch line (§17); built only if green at hour 20.
10. **Gemma 4 co-judge in the decision pipeline** (§9.4): a second, independent scorer (local Gemma 4 E4B, Apache-2.0, on-device) runs **in parallel** with the deterministic engine before the response returns. The response carries **both verdicts plus a combined verdict**. The deterministic engine remains authoritative and is what the golden-path test pins; the AI is a champion/challenger second opinion whose disagreement is surfaced for human review. Advisory, time-boxed, cuttable: if it times out or OOMs, the engine's decision returns unchanged.

(v1→v2 deltas retained: action-generalized kill-chain engine + 72h posture; IP & device in context with engine-owned observation store; P2P typed alias addressing + re-point signal; facts-and-timestamps contract; runtime state; salami closure; holds never auto-execute; idempotency.)

---

1. Problem & product

1.1 Problem

Banks make risk decisions on customer actions billions of times a day, and today those decisions are binary and context-blind. Authorized-push-payment (APP) scams defeat them structurally: the genuine customer, on genuine credentials, authorizes everything — including the *preparatory* steps. A coached victim doesn't just send a transfer; they first break the term deposit to free the funds, sometimes raise their daily limit, then add the beneficiary, then send. Each step looks individually unremarkable. **The sequence is the scam.** Meanwhile legitimate customers pay a permanent friction tax — SCA challenges and generic warnings on routine actions — which trains everyone to click through, so the warnings protect no one.

1.2 Product

Diakrisis is a decision API a bank calls before executing any customer-initiated action. It scores the action from explainable signals — counterparty intelligence, amount behavior, velocity, geo/device context, and kill-chain posture (what this account did in the last 72h) — and returns one of five graduated outcomes with a human-readable, pattern-specific explanation:

Outcome	Friction	Notes
`ALLOW`	None; SCA-exempt where regulation permits	The product working silently
`CONFIRM`	One specific sentence + one explicit tap (liquidity actions: a purpose prompt)	False-positive cost: one tap
`REQUIRE_APPROVAL`	A second authorized person must approve (four-eyes)	Initiator cannot approve their own action
`HOLD`	Cooling-off (30 min default) + scam-pattern-naming warning + one-tap cancel	Never auto-executes; explicit release
`BLOCK`	Stopped; manual review	Rare by design

It cuts both ways: risk up → friction up *with an explanation that names the pattern*; risk down → friction removed (PSD2 RTS Art. 18 TRA SCA exemption, basis logged per decision). The fraud tool that also removes friction.

1.3 Rubric mapping

Does It Work (40%) — live decisions across multiple action types, localhost, zero external dependencies, <50 ms. **Worth Building (35%)** — buyer: banks; APP liability shifting onto PSPs; dual value (fraud ↓, friction ↓); author builds these flows (beneficiaries, SCA, approvals) at a bank, in this same stack. **Pitch (15%)** — the kill-chain demo is a story. **Clever (10%)** — kill-chain posture + alias re-point + line quarantine + dual control. Not detection — *discernment*.

1.4 Constraints

Build 12:30 Sat → 12:30 Sun (confirm at induction); pitch 3 min + 2 Q&A. Solo (frontend seam §13). Demo offline-tolerant. Honesty explicitly scored: scope ledger and data-provenance statements written to be shown.

1.5 Scope ledger

Item	Depth
Event types: TRANSFER, P2P_TRANSFER, MASS_PAYMENT, TERM_DEPOSIT_BREAK, BENEFICIARY_ADD, LIMIT_CHANGE	Fully built
Berka/IEEE-CIS/ULB ingestion + feature store (Java ETL)	Fully built
Signal catalog (25 signals, §6) + one weight table	Fully built
Kill-chain posture (72h) + K signals; observation store (devices, IPs, alias resolutions)	Fully built
P2P alias addressing, resolution history, re-point detection	Fully built
Mass-payment batch engine: batch signals + per-line quarantine + partial acceptance	Fully built
Approval workflow (four-eyes): routing rules, approve/reject, initiator≠approver enforced	Fully built
Typologies (7 named patterns, §7)	Fully built
Policy bands + instant-rail appetite + SCA exemption + explanation templates (EN; EL stretch)	Fully built
ML subsystem — M1 Smile GradientTreeBoost: engineered features, time-split CV tuning, isotonic calibration, LR baseline, eval suite, importances, model card	Fully built, full treatment
Vector similarity — Qdrant exemplar store + M2 k-NN fraud-neighbor signal + similar-cases panel	Fully built (in-JVM KDTree fallback pre-decided)
Gemma 4 co-judge — parallel LLM scorer (E4B via Ollama), both verdicts + combined verdict in response; champion/challenger pattern	Built last, cut first — engine never depends on it
Cross-account counterparty reputation — shared store + X1 network signal + "flagged by another victim" warning	Fully built (cut-ordered after core demo)
ISO 20022 reason codes on every decision	Built (enum + field)
Feedback-loop counters (confirmed-save / false-positive) + self-improving narrative	Built (counters + slide arrow)
Vulnerability-aware friction (escalate one band for flagged-vulnerable accounts)	Documented / pitch line — build only if green
REST + WebSocket API; action lifecycle state machine	Fully built
Mini-bank UI (accounts, deposits, payees, transfer/P2P/batch flows, 5 intervention renderings, approver view)	Fully built
Ops dashboard (decision feed, signal breakdown, posture chips, exemption & saved counters, latency)	Fully built
SQLite persistence (features RO; decisions/audit/observations RW)	Built simple
GeoIP: local CIDR table (demo); MaxMind GeoLite2 (production)	Built simple
AI challenger/reviewer (Gemma 4 E4B, advisory ops-panel card)	Optional, post-decision, advisory-only — built last, cut first (§9.4)
Card actions, loan drawdowns, login risk, back-office case UI, gateway plugin, multi-currency	Documented only (§16)

1.6 Competitor comparison

The category is real and well-funded — which is market validation, not a threat (Visa acquired Featurespace; Mastercard is acquiring in the space; PSP liability for APP scams is shifting across Europe). Diakrisis is **not** trying to out-detect them; it occupies the layer they leave to the bank. Verified against current public positioning (never claim they are "black boxes" — Feedzai ships Whitebox Explanations and Visa A2A Protect ships explainable AI):

Capability	Featurespace / Visa A2A Protect	Feedzai	BioCatch	Diakrisis
Real-time risk score	Yes	Yes	Yes (behavioral biometrics)	Yes (deterministic + explainable)
Explainability	Analyst-facing	Whitebox, analyst-facing	Behavioral signals	**Customer-facing** — names the scam script to the victim at decision time
Owns the customer outcome	No — score to the bank	No — score to the bank	No	**Yes** — 5 graduated outcomes are the product
Friction *removal* (SCA exemption)	Implicit	"without unnecessary friction"	No	**Explicit** — `sca_exempt` + Art.18 basis logged per decision
Kill-chain / cross-action memory	Session/txn profiling	Txn profiling	Session behavior	**Yes** — 72h posture scores the sequence (deposit-break → drain)
Dual-control as a fraud outcome	No	No	No	**Yes** — REQUIRE_APPROVAL; works on a fully coached victim
Cross-account network effect	Within one tenant	Within one tenant	Consortium signals	**Yes** — shared counterparty reputation; warns victim 3 from victim 1
AI co-judge with bounded escalation	—	—	—	**Yes** — champion/challenger, capped one-band, deterministic
Target buyer	Tier-1 banks, Visa-scale	Tier-1 / enterprise	Large banks	**Mid-tier banks** — one drop-in API behind the gateway

The one-line position: *the incumbents detect and score for the fraud team; Diakrisis decides and speaks — graduated outcomes, victim-facing warnings, dual control, a kill-chain memory, and a cross-account moat — as one drop-in API a mid-tier bank can actually adopt.* Said first in the pitch, this turns "doesn't this exist?" into "yes, and Visa just paid for it — I build the layer they don't."

2. Domain model

```
ActionEvent
  event_id        String   (idempotency key — replay returns stored decision)
  account_id      String
  event_type      TRANSFER | P2P_TRANSFER | MASS_PAYMENT |
                  TERM_DEPOSIT_BREAK | BENEFICIARY_ADD | LIMIT_CHANGE
  payload         typed payload (below)
  context         SessionContext

TransferPayload                     // TRANSFER, P2P_TRANSFER
  counterparty    Counterparty
  amountEur       BigDecimal
  availableBalanceEur BigDecimal    // caller-supplied at initiation
  rail            SEPA | INSTANT | INTERNAL | P2P
  referenceText   String            // narrative only; no signal reads it

MassPaymentPayload                  // MASS_PAYMENT
  batchId         String
  purposeHint     PAYROLL | SUPPLIERS | OTHER
  items           List<BatchItem{itemId, counterparty, amountEur}>
  totalEur, availableBalanceEur, rail

DepositBreakPayload { depositId, principalEur, maturityDate, penaltyEur }
BeneficiaryAddPayload { counterparty }
LimitChangePayload { limitType, oldValueEur, newValueEur }

Counterparty                        // typed addressing — the P2P fix
  addressing      IBAN | ACCOUNT | MSISDN | EMAIL
  value           String            // "8829041-CY21" or "+35799123456"
  resolvedAccountRef String?        // alias/scheme directory lookup
  resolvedName    String?           // confirmation-of-payee
  displayName     String
  savedBeneficiaryId String?        // metadata only — NEVER the history key
  beneficiaryCreatedAt Instant?

SessionContext
  ts, sessionId, channel (WEB | MOBILE_APP)
  ip              String            // where the action was initiated from
  device          Device { deviceId, platform IOS|ANDROID|WEB, userAgent }
  // NO first-seen fields in the contract — the caller cannot reliably supply
  // them. The engine derives device/IP familiarity from its OWN observation
  // history: first sighting of (account, deviceId) or (account, ipCidr) IS
  // first-seen. Cold start: D/G weights ramp in over deployment warm-up;
  // production can seed from the bank's SCA trusted-device registry.

ObservationStore (persisted, engine-owned)
  (account, deviceId) -> firstSeen, lastSeen, platforms
  (account, ipCidr)   -> firstSeen, lastSeen
  (aliasKey)          -> resolution history          // feeds P1 re-point

AccountPosture (runtime, rolling 72h)
  fundsFreedEur72h · limitRaisedAt · beneficiariesAdded72h
  sessionIps · platformsSeen

Decision
  eventId
  engineVerdict { score 0–100, decision, scaExempt(+basis),
                  typologies[], signals[ {id, value, weight, contribution} ] }   // AUTHORITATIVE
  aiCoJudge { score 0–100, decision, reason: String,
              agreement: CONCUR | DIVERGE_STRICTER | DIVERGE_SOFTER,
              status: OK | TIMEOUT | UNAVAILABLE }?                              // ADVISORY, may be
absent
  combined { decision, basis: String, reasonCode: String }   // §8.3 reconciliation; reasonCode per
§8.4 (ISO 20022 + DKR scheme)
  lifecycle (endpoints relevant to the outcome; per-item array for batches)
  explanation { customer: String?, audit: String }
  latencyMs

  // explanation.audit is the canonical "why flagged" reason (deterministic, template-built
  //   from firing typology + ranked signal contributions); the ops "why" panel and the
```

```
//   GET /decisions/{id}/why endpoint render it. LLM (§9.4) only re-phrases, never sources it.
// INVARIANT: combined.decision === engineVerdict.decision. The AI never changes
// the outcome. "combined" exists to record concurrence and to RAISE a review flag
// when the two judges diverge — divergence is a feature (points a human at the
// edge case), never an automatic override. The golden-path test pins
// engineVerdict + combined; aiCoJudge is excluded from assertions (non-deterministic).
```

Identity rules (load-bearing): payment history keys on **resolved counterparty identity** (`resolvedAccountRef` when resolvable, else the alias key `(addressing, value)`) — one-offs, saved beneficiaries, and P2P-to-phone share one history. The contract carries **facts and timestamps only**; the engine derives every judgment (every "new", every age, every count, every familiarity).

---

3. Kill-chain model — actions, not transactions

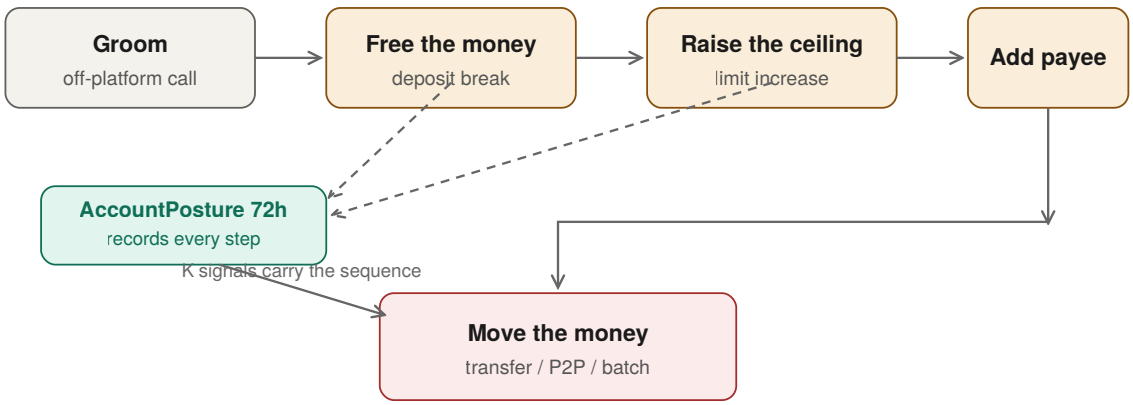


Figure 2 — The scam kill chain and account posture

```
groom (off-platform) → FREE THE MONEY → RAISE THE CEILING → ADD DESTINATION → MOVE THE MONEY
                       deposit break     limit increase      beneficiary add   transfer / P2P /
batch
```

Every step is scored at its own (mostly low) stakes **and recorded into AccountPosture**; K signals make the next money-moving action carry the sequence. Design choices: TERM_DEPOSIT_BREAK is never held or blocked — customer's money, customer's right; default CONFIRM with a **purpose prompt** ("are you doing this because someone asked you to move money?") — which is what well-run banks actually ask. BENEFICIARY_ADD alone: ALLOW or light CONFIRM (CoP mismatch raises it). LIMIT_CHANGE up: CONFIRM. All recorded. The demo's emotional center: the deposit break sails through with a gentle question; twenty minutes later the drain transfer slams into a HOLD *because the engine remembered*.

4. P2P — phone-number transfers

1. **Addressing is an alias** — history and signals key on resolved identity; first P2P to +357 99 123456 fires B1 even with no beneficiary record anywhere.
2. **Aliases re-point** — a number that previously resolved to account X and now resolves to Y is the SIM-swap/ hijack signature: signal **P1**, among the strongest in the table; fires on any amount (the signal is identity, not money).

3. **Instant is irrevocable** — on `rail = INSTANT | P2P`, effective band edges shift −8: a SEPA CONFIRM is an instant HOLD. Friction proportional to irreversibility.

Confirmation-of-payee runs on alias resolution (`resolvedName` vs expected) → B5.

4A. Mass payments — quarantine the line, not the payroll

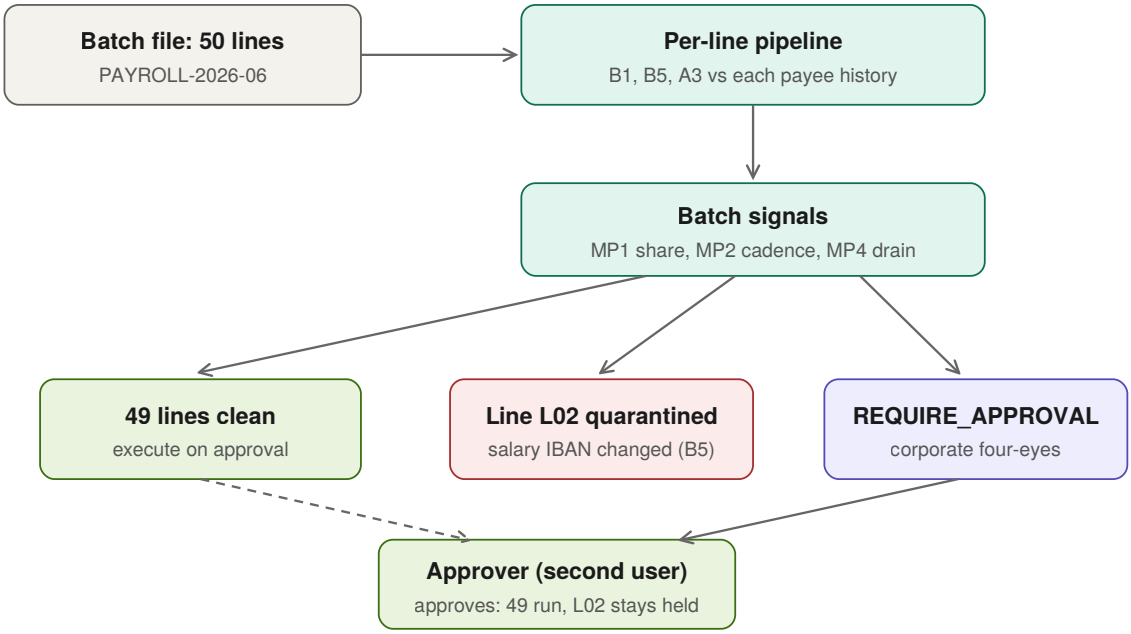


Figure 4 — Mass payment: two-level decision and line quarantine

Bulk files (payroll, supplier runs) are a business-account staple and a fraud vector twice over: **payroll redirection** (one changed IBAN among 50 salaries — BEC-style) and **mule fan-out** (a compromised account draining via 30 fresh counterparties in one file). Blocking an entire salary run over one bad line punishes 49 employees; v3 therefore decides at **two levels**:

- **Per line:** each item runs the per-transfer signals against *that counterparty's own history* (B1/B5/A3). Flagged lines are **quarantined** (item-level HOLD) while clean lines proceed — partial acceptance, exactly how real bulk processing should behave.
- **Per batch:** batch-level signals (below) can escalate the whole file — and on business accounts the batch routes through **REQUIRE_APPROVAL** (§8.2) regardless, matching real corporate dual-authorization.

ID	Batch signal	Weight	Logic
MP1	New-counterparty share	16	Fraction of lines with no history (payroll ≈ 0; fan-out ≈ 1)
MP2	Cadence/total anomaly	12	Batch total + day-of-month vs the account's batch history (payroll is rhythmic)
MP4	Batch drain	14	totalEur / availableBalance; interacts with K1

(MP3 isn't a new signal — it's the per-line application of B5/A3, which is what catches the redirected salary.)

Demo seed: the business demo account carries three months of payroll batches grouped from recurring real Berka payees (real payees, real amounts; the grouping is presentational — disclosed).

4B. Cross-account counterparty reputation — the network moat

Every signal so far protects the *sender*. But a scam has a *receiver* — the mule account — and that account is reused across victims within minutes. Diakrisis keeps an engine-owned **CounterpartyReputation** store: whenever any account's transfer to a counterparty is **HELD or BLOCKED** (or a hold is cancelled = confirmed scam), that resolved counterparty identity is marked, with a timestamp and decay.

Signal X1 (weight 20, decaying over hours): the destination counterparty was flagged on *another* account recently. It fires on the **third** victim before they finish typing — even if, for them, the payee is new and the amount unremarkable. No single-account, single-transaction tool can produce this; it requires the cross-account view, which is exactly the moat.

```
CounterpartyReputation (engine-owned, shared across accounts)
  resolvedCounterpartyId -> { flagCount, lastFlagAt, worstOutcome, decay }
  written on every HELD/BLOCKED/cancelled-hold decision
  read by X1 on every incoming transfer (any account)
```

Entrepreneurial framing (say it in the pitch): every bank running Diakrisis makes every *other* Diakrisis bank safer — classic network effect, the defensible moat an investor-minded judge rewards. In production this is a shared reputation service (privacy-preserving: hashed counterparty refs, no customer PII crosses the boundary — say this unprompted, it's the GDPR answer).

Demo (the "doesn't exist" moment): judge A's €4,000 transfer is HELD. Seconds later judge B (second browser) starts sending to the *same* account — before they finish, the screen warns *"This account was flagged 30 seconds ago in another scam attempt."* No consumer bank does this; every judge feels the absence instantly.

Cut order: built after the core demo (scenes 1–3) is green; X1 degrades to "off" cleanly (the store simply isn't consulted) with zero effect on every other signal.

5. Data subsystem

- **Berka (PKDD'99) — VERIFIED against the actual data (gate run pre-event on the Teradata GitHub mirror, 1,056,320 rows, dates remapped 2013–2018):**
- **Gate PASSED:** 208,283 outgoing transfers (operation `ROB`) with **100% counterparty bank+account coverage** — 6,064 distinct counterparties across 3,602 accounts. The composite-key fallback is retired.
- **History depth is demo-grade:** median 29 payments per account→counterparty pair, median 45 outgoing transfers per account, **528 accounts with 60+ transfers and 3+ payees** (top candidates: 7819, 3834, 8261). B1/B2/B4/A3 baselines are rich.
- **Balance on every transaction row** → A2 and available-balance realism are native.
- **5,542 perfectly rhythmic monthly pairs** (same day-of-month, near-zero amount variance — executed standing orders) plus the `order` table (6,471 standing orders) → MP2 cadence baselines and payroll-batch seeding come from real recurring behavior.
- **869 accounts have a real second authorized person** (`disp` table: OWNER + DISPONENT) → the approval demo runs on accounts that *genuinely* have dual access. The four-eyes flow isn't staged onto the data; it's in the data.
- Operation codes pre-translated by the mirror (ROB/WIC/CIC/COB/CCW); debit amounts sign-normalized; one disclosed EUR scaling constant.
- **Two honest constructions, disclosed:** Berka carries **no intraday timestamps** (dates only) and **no counterparty names or term deposits** — so (a) C1 is **weekday-pattern from Berka + hour-of-day learned at runtime** via the observation store, (b) CoP names and the demo term deposits are constructed on top of the real accounts, and said so.

- **IEEE-CIS (Kaggle)** — real labeled fraud: trains M1 on the feature intersection (amount, hour, velocity counts, recency deltas) via shared `Features.java`; time-based split; PR-AUC reported honestly.
- **ULB creditcard** — canonical imbalanced benchmark; one appendix number.
- **The honest line (say unprompted):** no public corpus of labeled APP-scam transfers, deposit-break sequences, alias hijacks, or payroll-redirection batches exists — banks cannot publish them. Legitimate-behavior baselines are real (Berka); the fraud-trained ML signal is real (IEEE-CIS); typologies and kill-chain rules are encoded from published scam patterns and regulator case categories. Demo scams are staged live against real histories. Same epistemic position as every vendor; the difference is saying it first.
- **GeoIP behind a `GeoResolver` interface (zero hot-path dependency):** the engine calls `GeoResolver.country(ip)`; the demo implementation is a local CIDR→country table seeded with the scripted IPs ("unknown" default), so nothing touches the network or a license server during the event. **Production swaps in the MaxMind GeoLite2-Country `.mmdb`** behind the same interface — a one-line config change, no engine code change. (MaxMind now requires an account, license key, signed EULA, and 90-day key reconfirmation, so it is deliberately *not* in the demo path; the interface seam is the right way to be production-ready without taking that dependency on. Optional: load the real Country mmdb locally tonight behind the same interface if desired — country is all G1 needs.)

6. Signal catalog — one weight table (`Weights.java`)

Signals are pure functions over (ActionEvent, FeatureStore, RuntimeState, Posture, Observations) → value ∈ [0,1]. Score = Σ weight·value, clipped 0–100. Hand-set starting weights by design (regulatory explainability; per-bank calibration roadmap); live-tunable on the ops panel.

ID	Signal	W	Fires on	Logic
B1	New counterparty	14	transfers, lines	No history on resolved identity
B2	Counterparty age (decay)	10	transfers	≈1.0 at minutes, ≈0.9 next day, →0 over ~90d
B3	Added this session	8	transfers	Creation within current sessionId
B4	History depth	−12	transfers	Deep history credits trust
B5	Name mismatch (CoP)	16	transfers, adds, lines	resolvedName vs expected
P1	Alias re-point	22	P2P	Alias resolves differently than its own history — SIM-swap tell
A1	Amount anomaly	12	transfers	Robust z vs account distribution; **logical amount**
A2	Balance drain	18	transfers	logicalAmount / available; >0.8 classic tell
A3	Counterparty-amount anomaly	12	transfers, lines	vs this counterparty's mean
A4	Threshold hugging	6	transfers	Just-under round limits
V1	Burst velocity	8	all	Actions/hour vs baseline (runtime)
V2	Escalation	10	transfers	Rising amounts, same counterparty
C1	Out-of-pattern time	6	all	Weekday habit from Berka baseline; hour habit runtime-learned (Berka has no intraday time)
C3	Retry pressure	8	transfers	Server-side raised-amount attempts per session
G1	Unfamiliar geo	12	all	IP country/CIDR vs observation store
G2	New network	6	all	New prefix, familiar country
D1	Device age (decay)	10	all	Observation store first sighting; ≈0 after 30–60d; warm-up ramp
D2	Platform anomaly	6	all	Android-only account suddenly WEB
K1	Funds freed recently	16	transfers, batches	fundsFreed72h covers this amount; 72h decay
K2	Limit raised recently	10	transfers	Within 72h, scaled by raise
K3	Beneficiary-add burst	8	transfers	≥2 adds in 72h
MP1	New-counterparty share	16	batches	§4A
MP2	Cadence/total anomaly	12	batches	§4A
MP4	Batch drain	14	batches	§4A
M1	Model score	18 cap	transfers, lines	Calibrated Smile GBT probability (isotonic)
M2	Fraud-neighbor share	12 cap	transfers, lines	Distance-weighted share of fraud among k-NN exemplars in Qdrant
X1	Cross-account reputation	20	transfers, P2P, lines	Destination flagged (HELD/BLOCKED/cancelled) on another account recently; hours decay — §4B network moat

Logical-amount rule (salami closure): A1–A3 evaluate `max(thisAmount, Σ 24h to same resolved counterparty)` from runtime state. **Cold start:** thin-history accounts down-weight behavioral signals, lean on typologies + M1 + G/D, widen bands.

7. Typologies — executable composites

Single match → outcome pinned to HOLD (floor and ceiling); two+ matches or raw ≥ 90 → BLOCK. Each carries its explanation template that *names the script*.

#	Typology	Rule (simplified)
1	Safe-account scam	B1 ∧ A2>0.7 ∧ (B3 ∨ D1>0.5 ∨ G1>0.5)
2	Invoice redirection	B5 ∧ A3 on established counterparty
3	Purchase scam	B1 ∧ rail∈{P2P,INSTANT} ∧ first-time modest amount
4	Romance / repeat victim	V2 across days ∧ B2>0.4
5	Liquidation kill-chain	K1>0.6 ∧ B1 ∧ A2>0.6
6	Payroll redirection	established batch pattern ∧ ≥1 line B5/changed-ref → quarantine line(s), batch proceeds (to approval)
7	Mule fan-out	MP1>0.7 ∧ MP4>0.6 → batch HOLD/BLOCK

8. Scoring, bands, co-judge reconciliation, approval, SCA

8.0 Canonical decision ordering (engine-first — never AI-first)

```
decide(event):
  engineVerdict = engine.score(event)          # REQUIRED, deterministic, authoritative
  aiVerdict     = aiCoJudge.score(event, engineVerdict.signals)   # BEST-EFFORT, ≤600ms, may be
absent
        || engine and AI run concurrently (StructuredTaskScope); engine result is the one that MUST
return
  combined      = applyCombineRule(engineVerdict, aiVerdict)      # DETERMINISTIC reconciliation
(§8.3)
  return { engineVerdict, aiVerdict, combined, ... }
```

The engine never waits on, nor depends on, the AI. If `aiVerdict` is absent (timeout/unavailable), `combined = engineVerdict` unchanged. The AI reads the engine's signal vector (so its second opinion is informed), which is why it is logically second even though it executes in parallel. **Order is always engine → AI → deterministic combine; never AI → engine.**

8.1 Bands

ALLOW 0–29 · CONFIRM 30–59 · HOLD 60–84 · BLOCK 85+. Instant/P2P rail: effective edges −8. Typology overrides per §7. Non-monetary actions cap at CONFIRM. On transfer ALLOW: TRA check → `scaExempt` + basis ("where regulation permits"; the bank's compliance function owns the fraud-rate attestation — the division of responsibility the RTS expects).

8.2 REQUIRE_APPROVAL — four-eyes dual control

Applied **after** banding, replacing the banded outcome (never downgrades BLOCK):

Trigger	Rule
Policy: corporate dual-auth	MASS_PAYMENT on business accounts → always (real corporate e-banking behavior)
Policy: big control changes	LIMIT_CHANGE raise > 2× → approval
Policy: dual-control threshold	TRANSFER > account's approval threshold (e.g. business €1k+)
Risk-routed	Score in HOLD band on accounts **with a designated approver / trusted contact** → approval *instead of* plain hold (retail vulnerable-customer pattern)

Mechanics: the action parks in `PENDING_APPROVAL`; a second authorized user approves or rejects; **initiator ≠ approver enforced (403)**; approvals expire (24h) to `EXPIRED`. Quarantined batch lines route their release through the approver too. **Why it matters for scams:** a coached victim will click through their own warnings — but cannot approve as the second person. Dual control breaks single-victim coercion structurally, not probabilistically.

8.3 Divergence-escalation policy — the co-judge can add friction, never block, never release

`applyCombineRule(engineVerdict, aiVerdict)` is a fixed, testable function. Default: `combined = engineVerdict`. Then exactly one adjustment may apply:

Case	aiVerdict	Action on `combined`
Concur	same band, or absent	none — engine stands
AI softer	`DIVERGE_SOFTER` (AI less worried)	**none** — never let the model talk the engine out of caution
AI stricter, low confidence	`DIVERGE_STRICTER`, score < `AI_ESCALATION_THRESHOLD`	none — only a `review_flag` on the ops feed
AI stricter, high confidence	`DIVERGE_STRICTER`, score ≥ `AI_ESCALATION_THRESHOLD` (=80, `Weights.java`)	**escalate exactly one band**: PASS→CONFIRM, CONFIRM→HOLD. Capped at HOLD — the AI can never force BLOCK

The asymmetry is the safety property, stated for the pitch: a high-confidence AI dissent *adds one notch of friction before the money moves* — so a scam the engine missed becomes a CONFIRM or HOLD in real time, not a post-hoc review (a flag after an irrevocable transfer is too late, and this policy exists precisely to fix that). But the escalation is capped at one step and at HOLD, so a *hallucinating* model can at worst cost a legitimate customer one extra tap or a 30-minute hold — recoverable friction — never a destroyed payment. **The AI can make the system more careful; it can never make it wrong, and it can never release.** The rule is deterministic on a recorded input (the AI's numeric score), so the golden-path test pins `combined` exactly (see T14, §12/§15); the model's text reasoning is shown to humans but never parsed into the decision.

`combined` records `basis` (e.g. "engine CONFIRM escalated to HOLD: AI co-judge DIVERGE_STRICTER @88") and, when escalation did *not* fire on a divergence, a `review_flag` so a human still sees it.

8.4 ISO 20022 reason codes

Every decision emits a standardized machine-readable reason code beside the human prose — the language bank integrations actually consume. Examples mapped from typologies/signals: `FRAD` (fraudulent), `FOCR` (following cancellation/return), `MS03` (reason not specified → never used; we always specify), plus an internal scheme tag (e.g. `DKR-SAFEACCT`, `DKR-KILLCHAIN`, `DKR-XACCT`, `DKR-INVOICE`). Field `reasonCode` on every `Decision`; the ops "why" panel shows it next to the prose. Cheap (an enum + a field) and it signals real bank-integration fluency to a banker judge.

9. ML subsystem — full treatment, pure JVM + vector store

9.1 M1 — supervised model

Smile `GradientTreeBoost` on the IEEE-CIS feature intersection, built as a real pipeline, not a placeholder: - **Features (`Features.java`, shared train+serve — skew impossible by compiler):** log-amount, hour/day cyclic encodings, velocity counts (C-columns ↔ runtime counts), recency deltas (D-columns ↔ time-since-last), amount-to-rolling-mean ratios, email-domain risk proxy at train time mapped to counterparty-type at serve time. ~14 engineered features. - **Training discipline:** time-based split (train on earlier, validate on later — no leakage); class imbalance via sample weights; hyperparameter search (trees, depth, shrinkage, subsample) with time-split CV; **isotonic calibration** on the validation fold so the output is a usable probability, not a rank hack. - **Baseline:** L2 logistic regression trained on the same vectors — reported next to the GBT (the comparison the academic expects). - **Evaluation suite (appendix slide + model card):** PR-AUC (primary, imbalance-appropriate), ROC-AUC, calibration curve, confusion matrices at the chosen operating points, and ULB benchmark run of the same pipeline. - **Explainability:** Smile feature importances surfaced live on the ops panel next to each M1 contribution — model-level "why" beside signal-level "why".

9.2 M2 — vector similarity over fraud exemplars (Qdrant)

Storage is hybrid by design: **SQLite for exact-key stores** (features, audit, observations — primary-key reads are not a similarity problem), **Qdrant for vectors**. The `etl` module embeds normalized `Features.java` vectors of IEEE-CIS labeled rows (fraud exemplars + a legitimate sample) into a Qdrant collection. At decision time the engine queries k-NN (k≈25, cosine) and computes **M2 = distance-weighted fraud share among neighbors**. Two products from one query: - a signal that catches fraud *shapes* the supervised model may rank poorly, and - **case-based explanation** on the ops dashboard: "this action most resembles 14 confirmed fraud cases (avg distance 0.12)" — retrieval-grounded explainability, which lands with both the academic and the AWS judge.

Fallback, pre-decided: if the Qdrant container misbehaves at the venue, Smile's in-JVM KDTree over the same vectors serves the identical signal with zero infrastructure — one interface (`ExemplarIndex`), two implementations, chosen by config flag.

9.4 AI challenger/reviewer — Gemma 4, optional, advisory-only (the "committee" layer)

A local **Gemma 4 E4B** (Apache 2.0, `ollama run gemma4`, on-device — "data never leaves the bank", a strong enterprise-AI answer for the AWS judge) acts as a **second-opinion reviewer beside the deterministic engine, never inside it.** Per flagged decision it renders a card on the ops dashboard: - **Engine score** — e.g. 84 / HOLD (computed, auditable — the ground truth) - **AI opinion** — concur · "go further" · "be gentler" (advisory) - **Reasoning** — plain language over the signal vector + typology

Hard boundary (the line that makes this score instead of backfire): the engine decides; the AI comments; the human reads both. The model's opinion is advisory and **never moves the score** — the deterministic verdict is what executes and what the golden-path test pins. This is a **model-vs-challenger** pattern from real risk ops, and it rhymes with the human four-eyes approval flow (§8.2): two reviewers, one accountable and one explanatory. -

Input: the computed `Decision` object only (the engine's *reasoning*, not raw customer money data — keeps the privacy story clean). - **Demo use:** a two-second glance — "our AI reviewer independently flags the same pattern and explains it" — never narrated at length (that would spend pitch time on the 10% axis). - **Q&A pre-load (a judge WILL ask "what if they disagree?"):** "The engine decides — auditable and regulator-defensible. The AI is a challenger that surfaces a second perspective for the analyst. Disagreement is a feature: it flags the edge cases a human should review." - **Cuttability:** runs alongside engine + Qdrant + two UIs on one laptop; cold inference is a flake risk. Built at ~hour 20 **only if green; first thing cut.** The deterministic templates (§11) render with or without it. Weights cached tonight (`ollama pull gemma4` — legal, public artifact).

9.3 Resilience principle (unchanged)

M1 and M2 are additive: weights → 0 leaves the deterministic engine and typologies fully functional. That is failure isolation, not simplicity — the demo's 40% never depends on a model or a container behaving. Domain-transfer caveat pre-loaded: card-fraud anomaly *shape* transfers; APP semantics live in typologies and the kill chain.

9.4 Gemma 4 co-judge — two verdicts, one authoritative

A second, independent scorer runs **in the decision pipeline, before the response returns**: local **Gemma 4 E4B** (Apache-2.0, on-device via Ollama — data never leaves the box, a clean enterprise-AI answer for the AWS judge). It is given the structured action plus the engine's signal vector and asked to return its own `{score, decision, reason}` as a fraud-risk judge.

Parallel, not serial. Engine and Gemma run concurrently (Java 26 structured concurrency, `StructuredTaskScope`); Gemma carries a hard time budget (≈600 ms). The engine's deterministic decision does **not** wait on the model beyond that budget.

Both verdicts return, plus a combined verdict: - `engineVerdict` — deterministic, auditable, **authoritative**. Sets the band, the outcome, the SCA decision. Pinned by the golden-path test. - `aiCoJudge` — the LLM's independent score + plain-language reason + an `agreement` flag (CONCUR / DIVERGE_STRICTER / DIVERGE_SOFTER). May be absent (`TIMEOUT/UNAVAILABLE`) with zero effect on the outcome. - `combined` — reconciled by the **deterministic §8.3 policy**: equals the engine's decision, except a high-confidence stricter AI dissent escalates it **one capped band** (PASS→CONFIRM→HOLD, never BLOCK, never softer). Records `basis` and any `review_flag`.

Why determinism survives even though the AI is in-path: the AI never *directly* sets the band. Its numeric score feeds the fixed §8.3 rule, which escalates at most one capped step. Because that rule is deterministic on a recorded number, the same request always yields the same `combined` — the golden-path test still pins T1–T14. The model's free-text reason is shown to humans, never parsed into the outcome. This is champion/challenger with a **bounded, one-directional, deterministic** escalation: the engine is champion of record; the AI challenger can add friction within a hard cap, never remove it, never hard-block.

Two surfaces consume it: the ops dashboard shows the co-judge card (both scores, the reason, the agreement badge) beside the live decision; the four-eyes approval screen (§8.2) shows the approver the AI's brief as a second opinion before *they* — a human — approve.

Cuttability (unchanged principle): Gemma sits behind an `AiCoJudge` interface; the no-op implementation returns `UNAVAILABLE` and the product is fully functional. It is built last (§15) and is the first thing cut. Engine, typologies, M1, M2 never depend on it.

Q&A line (hold it exactly): *"Does the AI decide? No. Two scorers run in parallel — a deterministic engine of record and a Gemma-4 co-judge. If the AI is more worried and highly confident, a fixed rule adds one notch of friction — a pass becomes a confirm or hold — before the money moves, so a missed scam is caught in real time. But it's capped: the AI can never hard-block and never release. A wrong AI costs one extra tap; a right AI stops a scam. We let the model make us more careful, never let it make us wrong."*

9.5 Feedback loop — the self-improving narrative

Decisions produce outcomes, and outcomes are training signal: - **Cancel a HELD transfer** → confirmed save (the system was right) — increment `confirmedSaves`, log the signal pattern as a positive example. - **Release after the hold expires** → likely false-positive (customer overrode) — increment `falsePositives`, log for weight recalibration. - **Approve/reject** in four-eyes → labeled outcome on the approver's judgment.

No live retraining in the 24h build — the point is to *show the loop exists*: two counters on the ops dashboard and a `decisions → outcomes → calibration` arrow on the architecture slide. This reframes Diakrisis from a static classifier into a per-bank system that gets better the more it runs — and it is the direct answer to "your weights are hand-set" (they are the starting point; the loop calibrates them per bank). Logged to the `outcomes` table; production wires it to the §16 SageMaker retraining schedule.

10. Architecture

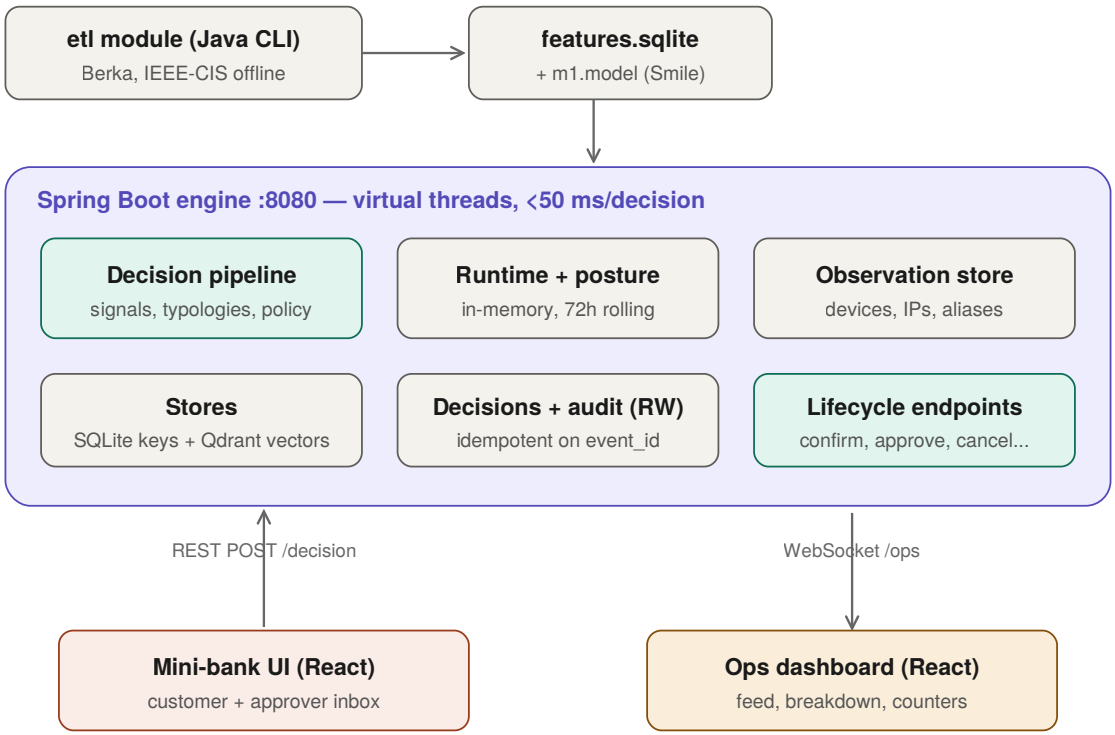
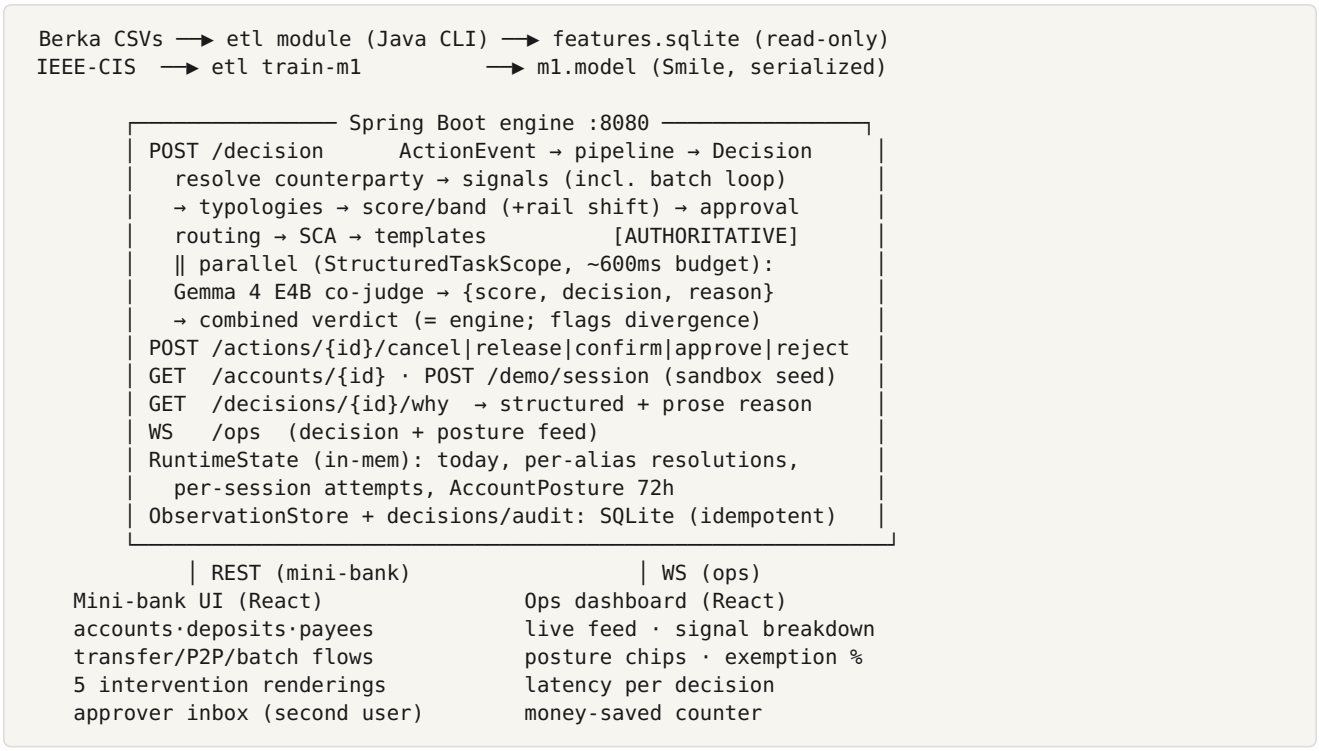
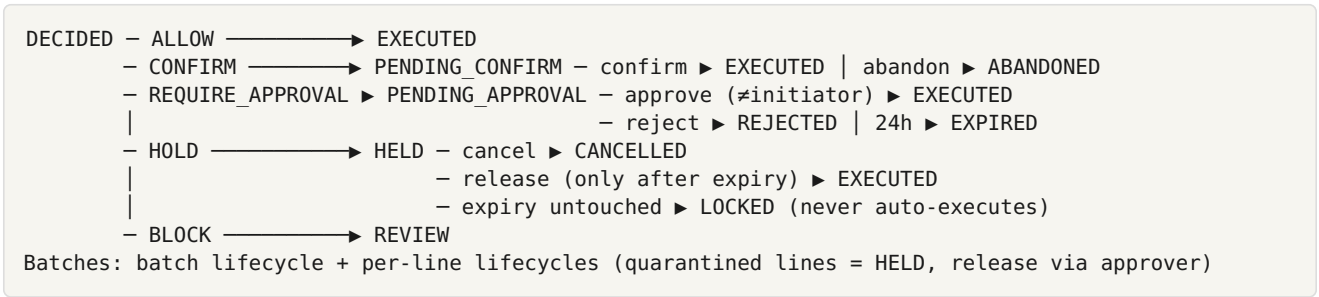


Figure 1 — System architecture



Latency budget <50 ms, printed per decision (virtual threads on; decision path is CPU-light lookups). No broker; WS fan-out in-process. Lifecycle state machine:



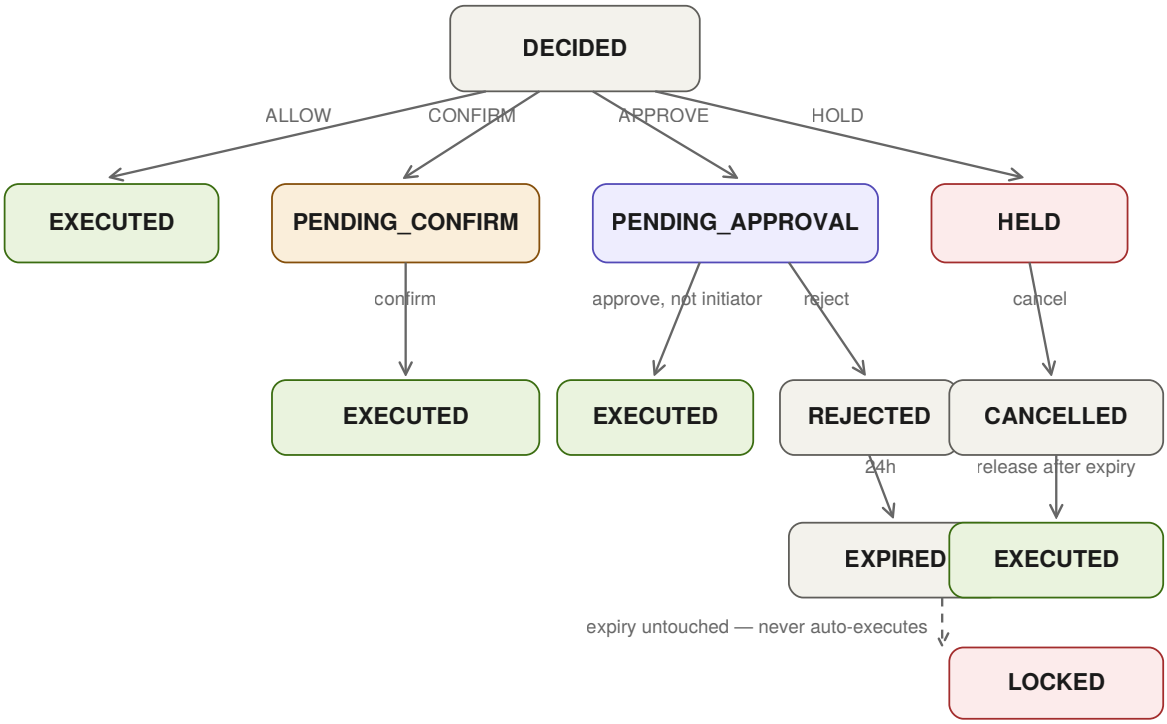


Figure 5 — Action lifecycle state machine (holds never auto-execute; initiator cannot approve)

11. API contract — full sample matrix

Shared shape as v2 §11 (all eleven samples carry over verbatim with Java-engine semantics unchanged). v3 adds:

11.12 MASS_PAYMENT → REQUIRE_APPROVAL + line quarantine (T11)

```
POST /decision
{ "event_id": "e-5801", "account_id": "biz-0042", "event_type": "MASS_PAYMENT",
  "payload": { "batch_id": "PAYROLL-2026-06", "purpose_hint": "PAYROLL",
    "items": [ { "item_id": "L01", "counterparty": { "addressing": "IBAN",
        "value": "CY11-....-771", "display_name": "E. Charalambous",
        "resolved_name": "E. CHARALAMBOUS" }, "amount_eur": 1420.00 },
      { "item_id": "L02", "counterparty": { "addressing": "IBAN",
        "value": "LT44-....-902", "display_name": "M. Ioannou",
        "resolved_name": "M. IOANNOU" }, "amount_eur": 1380.00 },
      "... 48 more lines ..." ],
    "total_eur": 68400.00, "available_balance_eur": 91200.00, "rail": "SEPA" },
  "context": { ...ctx } }
```

```
{ "event_id": "e-5801", "score": 31, "decision": "REQUIRE_APPROVAL",
  "approval": { "reason": "POLICY_CORPORATE_BATCH",
    "approve_endpoint": "/actions/e-5801/approve",
    "reject_endpoint": "/actions/e-5801/reject", "expires_in_hours": 24 },
  "typologies": [ {"id": "payroll_redirection", "lines": ["L02"]} ],
  "signals": [
    {"id": "MP1", "value": 0.02, "weight": 16, "contribution": 0.3},
    {"id": "MP2", "value": 0.10, "weight": 12, "contribution": 1.2} ],
  "items": [
    { "item_id": "L01", "decision": "ALLOW" },
    { "item_id": "L02", "decision": "HOLD", "signals": [
        {"id": "B5", "value": 1.0, "weight": 16, "contribution": 16.0,
         "detail": "M. Ioannou's salary IBAN changed since last month's batch"} ],
      "lifecycle": { "release_via": "approver" } },
    "... 48 × ALLOW ..." ],
  "explanation": {
    "customer": "Payroll batch of 50 queued for approval. 1 line quarantined: M. Ioannou's account
number changed since last month — verify with the employee before releasing that line.",
    "audit": "REQUIRE_APPROVAL (corporate batch); typology payroll_redirection on L02; 49 lines
clean." },
  "latency_ms": 41 }
```

Mule fan-out variant (T12): MP1 0.93 + MP4 0.81 → typology 7 → batch **HOLD/BLOCK** before approval is even offered.

11.12b Response carrying both verdicts + combined (T5b with co-judge)

```
{ "event_id": "e-5544",
  "engine_verdict": { "score": 84, "decision": "HOLD",
    "typologies": ["liquidation_kill_chain","safe_account_scam"],
    "signals": [ {"id":"K1","contribution":15.2}, {"id":"A2","contribution":15.5} ] },
  "ai_co_judge": { "score": 88, "decision": "HOLD",
    "agreement": "CONCUR", "status": "OK",
    "reason": "Funds freed from a deposit minutes ago are being sent almost entirely to a recipient
added moments ago — a classic liquidation-then-drain sequence. I would hold." },
  "combined": { "decision": "HOLD",
    "basis": "engine authoritative; AI co-judge concurs (88) — high-confidence hold" },
  "lifecycle": { "hold": { "duration_minutes": 30,
      "cancel_endpoint": "/actions/e-5544/cancel",
      "release_endpoint": "/actions/e-5544/release" } },
  "explanation": { "customer": "Stop. This matches how 'safe account' scams end ...",
    "audit": "HOLD: engine 84 + co-judge 88 CONCUR; combined HOLD." },
  "latency_ms": 41 }
```

Divergence example: if the engine says CONFIRM (52) and the co-judge says HOLD/DIVERGE_STRICTER, `combined.decision` is still **CONFIRM** (engine authoritative) but carries `"review_flag": "ai_stricter"` — surfaced on the ops feed for a human, never auto-applied.

11.13 Approval endpoints

```
POST /actions/e-5801/approve      (X-User: maria.cfo)
→ 200 { "event_id": "e-5801", "status": "EXECUTED",
        "items_executed": 49, "items_held": ["L02"] }

POST /actions/e-5801/approve      (X-User: same-as-initiator)
→ 403 { "error": "SELF_APPROVAL_FORBIDDEN" }

POST /actions/e-5801/reject       (X-User: maria.cfo)
→ 200 { "status": "REJECTED" }
```

Contract invariants unchanged: facts and timestamps in; every judgment derived; history keys on resolved identity; `explanation.customer` null exactly when the product works silently; golden path pins every sample.

12. Demo script (3:00) & catalog

1. **(0:25) The glide** — bill, ALLOW, SCA-exempt, exemption counter ticks. "We just *removed* friction."
2. **(1:05) The kill chain** — break deposit (CONFIRM + purpose prompt; posture chip lights) → add payee → send €4,850 → HOLD names the whole sequence; cancel; saved-counter ticks.
3. **(0:40) The breadth + the moat** — payroll batch: REQUIRE_APPROVAL, one line quarantined; approve from the phone as the second user. Then the "doesn't exist" beat: a second device sends to the account just held on the first → *"flagged 30 seconds ago in another scam attempt."* "Every bank running this makes every other safer."
4. **(0:50) Who pays + close** — liability shift; dual value; "I build these flows at a bank, in this stack — this drops in at action initiation behind the gateway."

#	Action	Verdict	Key signals/ typology	Outcome
T1	Bill, payee paid 41×	Legit	B4	ALLOW + exempt
T2	Rent, standing payee	Legit	B4	ALLOW + exempt
T3	€700 to €120-payee	Legit, anomalous	A3	CONFIRM
T4	Known supplier, changed ref	Fraud (Ty2)	B5+A3	HOLD
T5a	Deposit break €5,000	Prep step	D1+G2; posture	CONFIRM + purpose
T5b	€4,850 to 4-min payee, 20 min later	Fraud (Ty5+1)	K1+B1+B2+B3+A2	HOLD
T6	Escalating to 10-day payee	Fraud (Ty4)	V2+B2	HOLD
T7	First P2P to MSISDN	Unknown	B1, rail shift	CONFIRM + CoP
T8	Alias re-points	Fraud	P1	HOLD (any amount)
T9	3×€850 slices, 2h	Fraud	logical amount + A2	slice 3 HOLD
T10	New device+payee+drain+retries+foreign IP	Fraud (stacked)	B1+A2+G1+D1+C3	BLOCK
T11	Payroll 50 lines, 1 redirected	Fraud in batch (Ty6)	MP-clean + L02 B5	REQUIRE_APPROVAL, L02 quarantined
T12	30-line fan-out drain	Fraud (Ty7)	MP1+MP4	BLOCK
T13	Self-approval attempt	Control test	—	403
T14	Engine CONFIRM (52) + AI co-judge DIVERGE_STRICTER @88	Co-judge escalation	§8.3 rule	combined = HOLD (one-band escalation, capped)
T15	Account B sends to a counterparty HELD on account A 30s earlier	Cross-account moat	X1	HOLD — "flagged by another victim" warning

Fallbacks: live → scripted click-through → recording (Sunday 10:00). All scenes at real wall-clock time (C1 ≈ 0; typologies carry; showing the `ts` field if asked = explainability flex).

13. UIs

Mini-bank: account view (balance, deposits, payees), transfer/P2P/batch upload flows, deposit-break with purpose prompt, the five intervention renderings, **approver inbox** (login as second user; demo: approve from a phone). Ops: live feed, signal breakdown, posture chips, exemption-rate + money-saved counters, latency. **"Why was this flagged?" panel** — clicking any decision in the feed opens a plain-language explanation of *why* the engine reached its outcome: the firing typology named in words, the top contributing signals ranked with their euro/behavioral meaning ("freed €5,000 20 min ago → K1; sending 86% of balance → A2; payee 4 min old → B2"), the M2 similar-cases ("resembles 14 confirmed fraud cases"), and — when present — the Gemma co-judge's concurrence/dissent and reason. This is the deterministic audit reason rendered for a human; the optional LLM narrative (§9.4) enriches the phrasing but the structured reason underneath is template-driven and always available. Per-session sandbox seeding so judges' phones can't collide with the projector. Dark, product-grade. Teammate seam: entire frontend vs §11.

14. Stack — explicit (Java)

Layer	Choice
Language/runtime	**Java 26 if clean on the build machine tonight, else 25 LTS** — verify locally; zero design impact
Framework	Spring Boot 3.5.x (web, validation, WebSocket); virtual threads enabled
Build	Maven, multi-module: `etl/` (CLI: Berka→features.sqlite, IEEE-CIS→m1.model) + `engine/` (the service)
JSON	Jackson (records as DTOs)
ML	**Smile** `GradientTreeBoost` — pure JVM; shared `Features.java` train+serve
Persistence (keys)	SQLite via xerial `sqlite-jdbc` + Spring `JdbcTemplate` (no JPA — speed and control)
Vector store	**Qdrant** (single Docker container, official Java client, gRPC :6334) — fraud-exemplar k-NN; in-JVM Smile KDTree fallback behind `ExemplarIndex`
CSV/ETL	commons-csv in `etl` module
Tests	JUnit 5 + MockMvc; golden path = full T-catalog asserted
Frontend	React 18 + TypeScript + Vite + Tailwind + Recharts (unchanged)
Tooling	Claude Code; git live commits
Demo runtime	`java -jar engine.jar` + `vite` — two localhost processes
Tonight's runway (critical)	Spring Initializr skeleton + `mvn dependency:go-offline` + smoke build; `docker pull qdrant/qdrant`; `ollama pull gemma4` (E4B weights cached) — nothing touches the network during the event

15. Testing & build sequence

Tests: per-signal fixtures (hand-computed histories); typology scenario tests; **golden path = T1–T15 through POST /decision asserting exact outcomes** incl. salami, idempotent replay, hold release-only-after-expiry, P1 re-point, line quarantine, self-approval 403, **co-judge divergence-escalation (T14)**, and **cross-account reputation propagation (T15: flag on account A → X1 fires on account B)**. Run before every commit after hour 12. Malformed input → 400/422, never 500 (Bean Validation).

Hours	Deliverable	Axis
0–1	Repo, modules, domain records, `Weights.java` (coverage gate already passed pre-event)	40
1–3	etl: features.sqlite (payee tables, baselines, batch history); **first light:** POST /decision returns a scored decision on real history	40
3–6	Signals B/A/V/C + runtime state + logical amount + fixtures	40
6–8	G/D/K + observation store + posture + P2P alias + P1	40+10
8–10	Typologies + bands + rail shift + approval routing + SCA + templates + lifecycle SM	40
10–12	Mass-payment batch pipeline (per-line loop + MP signals + quarantine)	40+10
12–14	Mini-bank UI: all flows incl. batch upload + approver inbox + 5 renderings	40+15
14–16	Ops dashboard + WS + counters + posture chips	40
16–19	ML subsystem: feature engineering, tuned+calibrated M1, LR baseline, eval suite; Qdrant load + M2 + similar-cases panel	10+Q&A
10–12	(with batch) ISO 20022 reasonCode enum + field; feedback counters wired to cancel/release	35+10
19–20	Cross-account reputation store + X1 + "flagged by another victim" warning (ONLY after scenes 1–3 green)	10 (moat)
20–21	Gemma 4 co-judge (ONLY if green): Ollama wire, StructuredTaskScope parallel call, both-verdicts response, ops co-judge card	10
19–21	Sandbox sessions, hardening, golden path green, latency print, EL templates (stretch)	40
21–22	Polish; record fallback	40+15
22–24	Deck ≤5 slides; rehearse ×3 to 2:45; sleep 90 min if green	15

Cut order if behind: **Gemma 4 challenger (§9.4) → EL templates** → D2/G2/K3 → MP2 + T12 → ops charts → M2/Qdrant (KDTree keeps the signal) → typologies 3–4 → M1 tuning depth (model still ships). The challenger is the designated first sacrifice — plan around it at zero. **Never cut:** runtime state, K1, B1/B2/A2, typologies 1+5+6, approval flow, kill-chain + batch demo scenes, golden path. Canary: first light by hour 3 or cut M1 + ops charts immediately.

16. AWS production path (Botis Q&A)

Spring Boot container on **ECS Fargate** behind ALB (or **Lambda SnapStart** — the Java-on-Lambda answer he'll appreciate) · DynamoDB feature store + posture (**TTL=72h items map 1:1 to posture decay** — a tidy answer) · Kinesis → S3 immutable audit · API GW WebSocket ops feed · SageMaker or scheduled Fargate task for retraining, EventBridge cron · per-bank stages + KMS · vectors on **Amazon OpenSearch (k-NN)** or Aurora **pgvector** (Qdrant Cloud as the lift-and-shift option) · MaxMind on the container layer. One appendix slide; speak only when asked.

17. Pitch skeleton & Q&A pre-loads

Open: "Scam victims authorize everything themselves — including the steps *before* the payment. The scam isn't a transaction; it's a sequence. Banks score transactions." Demo (§12). Who pays: PSP liability shift; dual value; day-job integration sentence. Next: gateway-plugin pilot, per-bank calibration, outcome feedback loop. **Uniqueness vs incumbents (verified against the live market — never say "black box"; Feedzai markets Whitebox Explanations and Visa A2A Protect ships explainable AI):** (1) we own the **outcome**, not the score — incumbents end at a score+explanation for the fraud team; our product is the customer-facing decision itself (5

outcomes, script-naming warnings to the victim, purpose prompts, never-auto-execute holds); (2) **dual control as a fraud outcome** — REQUIRE_APPROVAL lives in the e-banking workflow layer incumbents don't own; the only mechanism that works on a fully coached victim; (3) **friction removal with the law in the payload** — `sca_exempt: true, basis: Art.18 TRA` as a logged decision; the only fraud tool that pays for itself twice; (4) **kill-chain as the unit of analysis** — "the scam isn't a transaction, it's a sequence; banks score transactions"; (5) **buyable shape** — one drop-in API for mid-tier banks vs Visa-scale enterprise cycles; (6) **network moat** — cross-account reputation means every Diakrisis bank makes every other safer, a defensible network effect incumbents' single-tenant deployments don't have; (7) **self-improving** — the cancel/release feedback loop calibrates per bank. Roadmap line ready: **vulnerability-aware friction** (regulator-driven — flagged-vulnerable accounts escalate one band; built if green, else spoken). Market validation said FIRST in the pitch: Visa bought Featurespace outright; Mastercard is acquiring in the space; PSP liability is landing across Europe — we build the layer the giants are buying toward, for banks they don't serve. Pre-loads: incumbents (use the uniqueness block above verbatim) · data provenance (§5, unprompted) · coached victim (time + naming the script + **dual control breaks single-victim coercion**) · hand-set weights (regulatory explainability; live-tunable) · M1 domain transfer · TRA nuance (bank owns attestation) · GDPR (legitimate interest; explainability is an Art. 22 asset) · cold start (warm-up ramp; SCA registry seeding) · why Java (the stack banks run; the author runs it at one).

18. Risks

Risk	Mitigation
~~Berka counterparty coverage~~	**RETIRED — gate run pre-event: 100% coverage on 208k ROB rows**
Maven/network at venue	`dependency:go-offline` + smoke build tonight; never touch network during event
Java 26 toolchain surprise	Verify tonight; 25 LTS fallback is a one-line property change
Kill-chain timing compressed in demo	Posture decay parameterized; break at minute 1, drain at minute 2 — K1 fires on any recency; "production window is 72h"
Batch pipeline eats the morning	It's a loop over the per-transfer pipeline + 3 signals; if hour 12 arrives without it, demo falls back to T1–T10 and batch goes documented-only (cut line pre-decided)
Qdrant container fails at venue	`ExemplarIndex` interface, KDTree impl behind a config flag — identical signal, zero infra; image pulled tonight
Gemma co-judge slow/ OOM in hot path	Hard ~600ms budget + parallel exec; `AiCoJudge` no-op fallback returns UNAVAILABLE; engine decision unaffected; first in cut order; weights pulled tonight
Score drift from §6	Single `Weights.java` constant; golden path pins outcomes
"Another fraud project"	Open by removing friction; kill-chain + quarantine + dual control are the stories no one else has

---

Diakrisis v4 · iFX Hack 2026 · A. Eftychiou